Arquitectura profesional para proyectos de ciencia de datos

Guía didáctica · Estudio de caso: Fase 5 — Research_AI_law

Audiencia: Data scientists junior / trainees

Objetivo: entender por qué un proyecto profesional se organiza en carpetas separadas (config, src, tests, outputs, notebooks) y qué principios de ingeniería respaldan esa decisión.

Estudio de caso: el proyecto Research_AI_law (estudio sobre regulación de IA para informar al Senado de Chile) implementa esta arquitectura. Cada concepto se conecta a archivos reales del proyecto.

Pre-requisito: haber leído conversación previa con Claude sobre Fase 5 v2.0.

Documento generado a partir del diálogo entre Pablo (data scientist en formación) y Claude (asistente de ciencia de datos), 2026

1. La pregunta original

Esta guía nace de una pregunta concreta:

> *"¿Por qué es una buena práctica la arquitectura de la Fase 5 para un proyecto de ciencia de datos? Es decir, por qué hay una carpeta config, una notebook, una outputs, una src, una tests... y qué hace cada una. Explícamelo todo en base a buenas prácticas profesionales."*

Por qué importa esta pregunta

En ciencia de datos profesional —especialmente para proyectos que se entregan a stakeholders externos como un gobierno, un regulador o un comité académico— el código y los datos viven en niveles de confianza distintos. Mezclarlos en un solo lugar (lo típico del 'notebook único de 2,000 líneas') rompe tres propiedades que son innegociables:

Reproducibilidad. Mismo input + mismo código = mismo output, siempre. El Senado debe poder re-ejecutar tu análisis y obtener exactamente las mismas cifras.

Auditabilidad. Cada decisión y cada celda del resultado son trazables a su fuente. Un revisor metodológico tiene que poder preguntar '¿de dónde sale este 0.234?' y obtener una respuesta verificable.

Mantenibilidad. Otra persona puede leer, modificar y extender el proyecto sin romperlo. Tu trabajo no muere cuando termines la tesis — alguien tiene que poder iterar sobre él.

La estructura de Fase 5 implementa estas tres propiedades vía **6 carpetas con responsabilidades distintas**. Cada una resuelve un problema específico que un científico de datos profesional enfrenta tarde o temprano.

2. El principio rector: separación de responsabilidades

Antes de explicar qué hace cada carpeta, conviene ver el contraste entre lo que ocurre cuando NO se separa (notebook monolítico) vs cuando SÍ se separa (arquitectura modular como tu Fase 5):

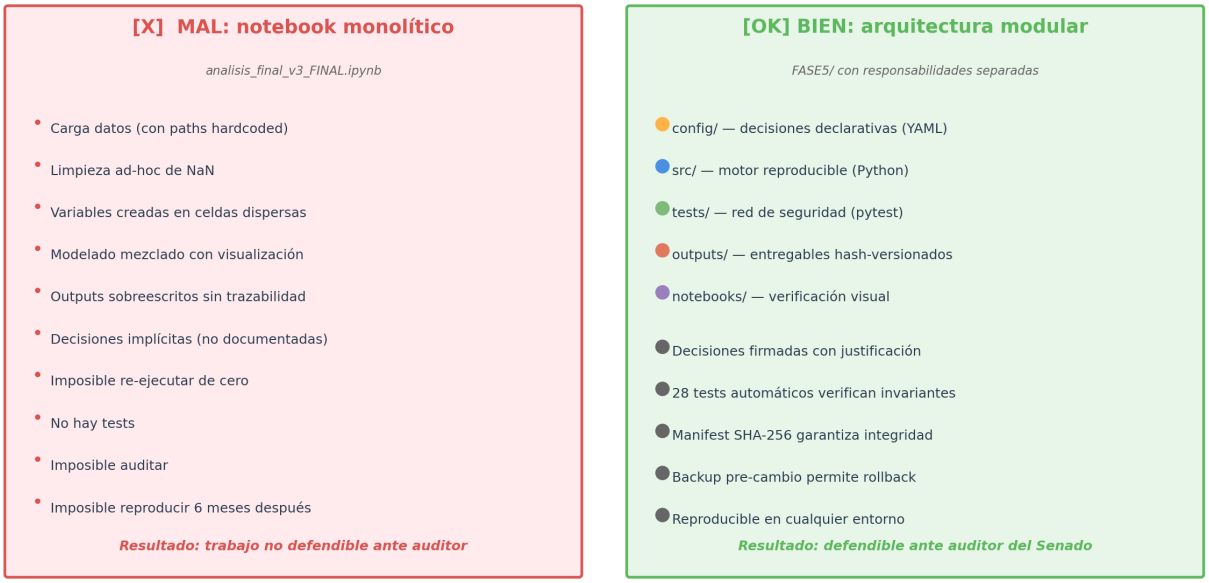

Figura 1. Contraste entre arquitectura monolítica y modular. La columna izquierda es el patrón típico de notebook único; la derecha es lo que tu Fase 5 implementa.

Lectura del diagrama

El problema central del notebook monolítico no es estético: es **operacional**. Los notebooks ejecutan celdas en orden no-lineal (podés correr la celda 5, luego la 3, luego volver a la 5), y el output depende de qué celdas corriste antes. Eso destruye la reproducibilidad. Un script `.py` se ejecuta de arriba abajo, una sola vez.

Además, los notebooks no se testean fácilmente y se rompen al hacer diff en git (cambian timestamps de ejecución y números de celda, generando ruido en los commits).

> **Regla profesional:** el código que produce resultados oficiales nunca vive en notebooks. Vive en `src/`. Los notebooks son para visualizar resultados, no para producirlos.

3. Vista general de la arquitectura

Tu Fase 5 organiza el trabajo en seis bloques con responsabilidades completamente separadas. Cada uno responde a una pregunta distinta:

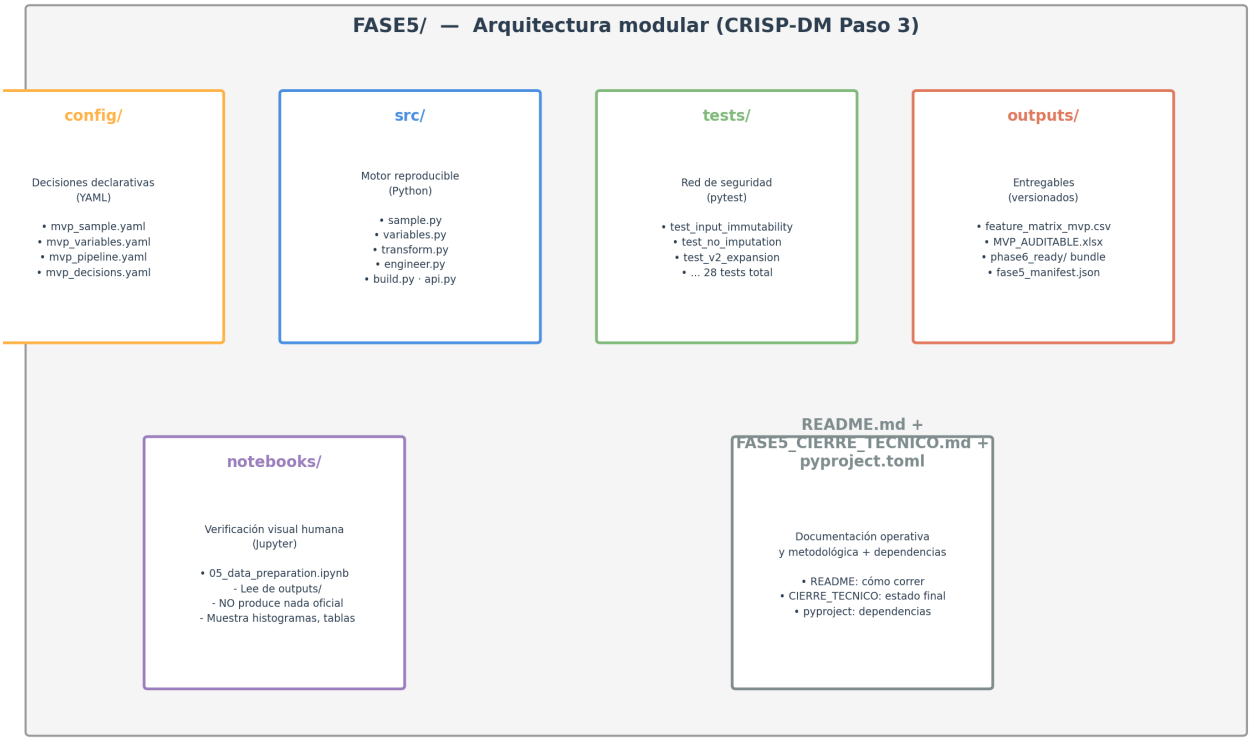

Figura 2. Arquitectura de Fase 5: cada carpeta tiene una responsabilidad única y un color asignado que mantendremos a lo largo de la guía.

Carpeta	Responde a la pregunta...	Tipo de archivo
config/	¿Qué decisiones se tomaron?	.yaml
src/	¿Cómo se transforman los datos?	.py
tests/	¿Cómo sé que sigue funcionando?	.py (pytest)
outputs/	¿Qué entregamos?	.csv, .xlsx, .json
notebooks/	¿Cómo verifico visualmente?	.ipynb
raíz	¿Cómo se usa esto?	README.md, pyproject.toml

En las siguientes páginas vamos carpeta por carpeta, conectando lo que hace con principios de ingeniería de software profesional.

¿Qué hay dentro?

```
FASE5/config/
  mvp_sample.yaml ← los 43 países preregistrados
  mvp_variables.yaml ← las 46 variables y sus roles
  mvp_pipeline.yaml ← transformaciones (log, z-score)
  mvp_decisions.yaml ← log de decisiones firmadas
```

Principio: 'configuration as code'

Las decisiones del proyecto se almacenan en archivos de datos (YAML), no hardcoded en Python. Esto es un patrón estándar de la industria (Twelve-Factor App, factor III). Los beneficios son cuatro:

Preregistro científico. En investigación seria existe la práctica del *preregistration*: declarar qué vas a hacer ANTES de mirar los resultados, para evitar p-hacking. Tu `mvp_sample.yaml` y `mvp_variables.yaml` cumplen exactamente esa función.

Decisiones audit-friendly. Cuando un revisor pregunta '¿por qué se incluyó Costa Rica pero no República Dominicana?', la respuesta es **una línea YAML con su** `razon`. No tienes que excavar en código.

Cambios sin tocar código. Si mañana decides agregar una variable, editas el YAML — no escribes Python nuevo. El `src/` ya sabe leer del YAML genéricamente.

Diff legible en git. Un cambio de configuración aparece como '+1 línea YAML' en un commit, no como un parche dentro de una función Python densa. Esto es enorme para revisión por pares.

Ejemplo real de tu proyecto

Esta es una entrada real de tu `mvp_decisions.yaml`:

```yaml
- id: F5-V2-004
  fecha: '2026-05-07'
  decision: 'No se incorpora reduccion dimensional (PCA) en Fase 5'
  justificacion: 'Fase 4 ya contiene analisis exploratorio...'
  revisor: 'investigador_principal'
  estado: approved
```

Si dentro de un año un revisor pregunta '¿por qué no usaron PCA?', la respuesta está aquí, firmada y fechada. Eso es **auditabilidad real**.

¿Qué hay dentro?

```
FASE5/src/
  sample.py ← filtra wide a 43 paises
  variables.py ← selecciona 46 vars del config
  transform.py ← aplica log + z-score robusto
  engineer.py ← features agregadas (n_binding, etc)
  build.py ← orquestador que llama a los anteriores
  validate.py ← checks de integridad
  audit_excel.py ← genera el Excel auditable
  phase6_bundle.py ← empaqueta el bundle para Fase 6
  api.py ← interfaz publica estable
```

Principio: Single Responsibility Principle (SRP)

Cada archivo de `src/` hace UNA cosa bien. Esto se llama *Single Responsibility Principle* (Robert C. Martin, 'Clean Code'). Cuando algo se rompe, sabes EXACTAMENTE dónde mirar — no buceas en un script de 1500 líneas.

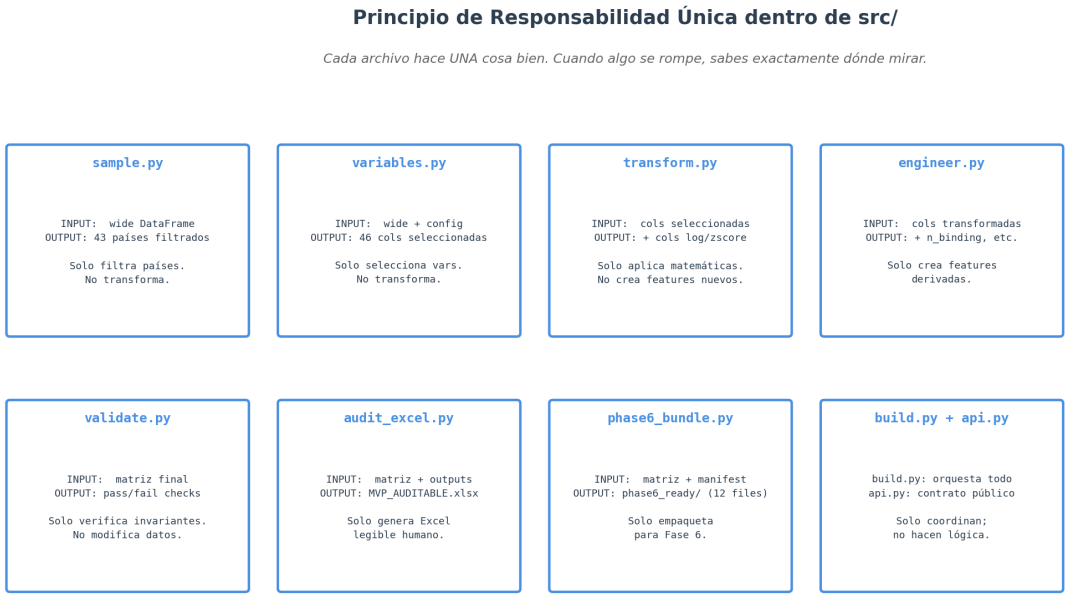

Figura 3. Cada módulo de src/ tiene un input y output bien definidos. La cadena de transformaciones es lineal y predecible.

El módulo especial: api.py

Este es un patrón de **encapsulación**. `api.py` define qué funciones son la interfaz 'pública' — qué puede usar Fase 6, Fase 7, Fase 8. Todo lo demás es interno.

Si mañana refactorizas `transform.py`, las otras fases no se rompen siempre que `api.py` mantenga su firma. Es **el contrato de Fase 5 con el resto del mundo**.

> **Analogía:** imagínate que `src/` es un restaurante. Los chefs (los módulos internos) cocinan en la cocina; el cliente (Fase 6) solo ve el menú (api.py). Si el chef cambia la receta internamente pero el plato sigue saliendo igual, el cliente no se entera. Eso es encapsulación.

¿Qué hay dentro?

```
FASE5/tests/
  test_input_immutability.py ← Fase 3 no se modifica
  test_sample_integrity.py ← los 43 paises son exactos
  test_variables_integrity.py ← las 46 vars existen en Fase 3
  test_no_imputation.py ← los NaN se preservan
  test_outputs.py ← feature_matrix tiene forma esperada
  test_phase6_ready_bundle.py ← el bundle es consumible
  test_api_and_validate.py ← el API funciona
  test_v2_expansion.py ← 8 invariantes de v2.0
```

Por qué los tests son innegociables

Tests = especificación ejecutable. Cada test es una afirmación verificable de algo que tu proyecto promete. `test_v1_vars_preserved_byte_for_byte` no es solo código — es un **contrato firmado** que dice 'las 40 variables v1.0 nunca cambiarán de valor sin que lo notes'.

Tests previenen regresiones. Auditaste tu Fase 5 v2.0 hoy. Mañana alguien cambia `transform.py` y accidentalmente rompe los z-scores. Sin tests: nadie se entera hasta que el reporte ejecutivo muestra cifras raras. Con tests: `pytest` falla en CI, el commit no se mergea.

Tests documentan invariantes. Cuando lees `test_no_imputation.py`, aprendes algo del proyecto sin tener que leer todo el código: 'ah, este proyecto no imputa, eso es decisión metodológica'. El test es **documentación viva**.

Tests dan confianza para refactorizar. Sin tests, refactorizar es ruleta rusa. Con 28 tests pasando, podés reescribir la mitad de `src/` y saber inmediatamente si rompiste algo.

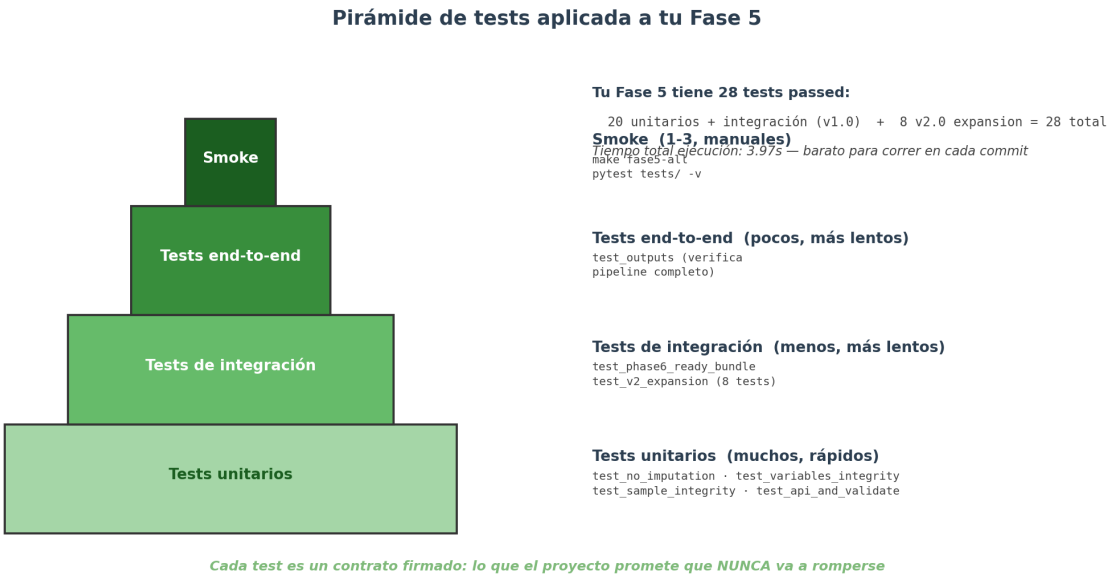

Figura 4. La pirámide de tests (Cohn, 2009): muchos tests unitarios rápidos en la base, menos tests de integración, pocos end-to-end. Tu Fase 5 sigue exactamente este patrón.

¿Qué hay dentro?

```
FASE5/outputs/
  feature_matrix_mvp.csv ← la matriz para modelado
  MVP_AUDITABLE.xlsx ← Excel para humanos no tecnicos
  coverage_report_mvp.csv ← diagnostico de cobertura
  fase5_manifest.json ← hashes SHA-256 de inputs/outputs
  phase6_ready/ ← bundle para Fase 6 (12 archivos)
  outputs.v1.0.backup/ ← snapshot pre-update (rollback)
```

El concepto crítico: manifest con SHA-256

El problema: ¿cómo sabes que el CSV que estás leyendo es el mismo que produjiste hace tres meses? La respuesta es **hashing criptográfico**:

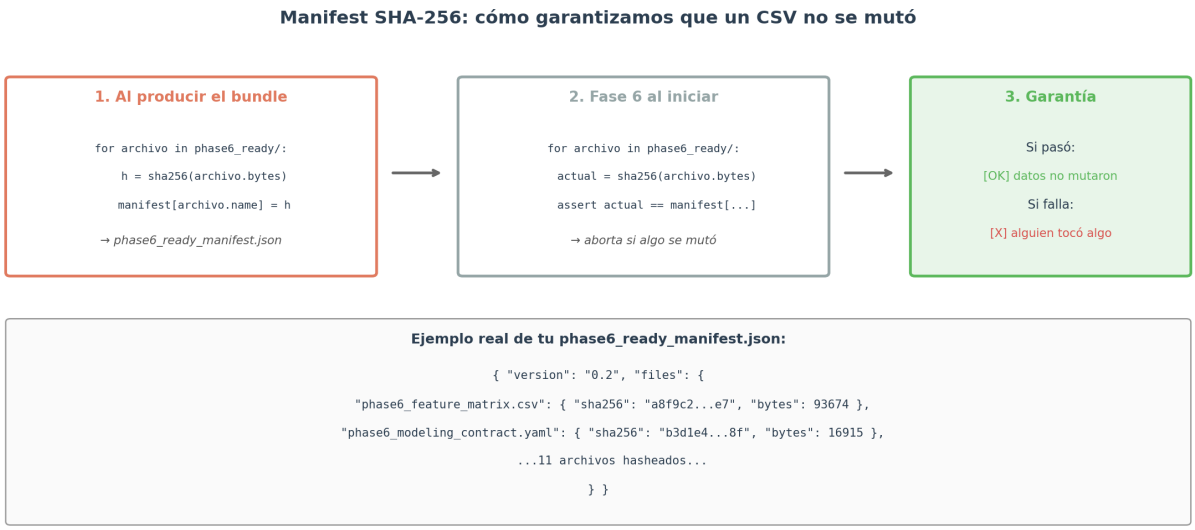

Figura 5. Mecanismo de manifest con SHA-256: si un solo byte del archivo cambia, el hash cambia completamente. Fase 6 verifica los hashes ANTES de comenzar a modelar.

Otros principios profesionales aplicados

Separación output humano / output máquina. `MVP_AUDITABLE.xlsx` es para que el jefe lo abra en Excel. `phase6_feature_matrix.csv` es para que Fase 6 lo lea con pandas. Mismos datos, dos formas — porque las audiencias son distintas.

Backup pre-update. La carpeta `outputs.v1.0.backup/` es un patrón de *rollback safety*. Si v2.0 sale mal por algún motivo, vuelves a v1.0 con un `mv`. Esto es exactamente lo que hacen sistemas de bases de datos profesionales.

Bundle como contrato. `phase6_ready/` no es solo 'outputs sueltos'. Es un **paquete con esquema explícito** (`phase6_modeling_contract.yaml`) que dice exactamente qué es cada archivo y cómo se consume. Esto es **API design para datos**, equivalente a definir un endpoint REST con OpenAPI.

¿Cuál es el rol legítimo de un notebook?

Hay una **regla de oro** moderna en ML engineering: *los notebooks NO ejecutan lógica de producción, solo la visualizan*. En tu Fase 5 esto significa:

El notebook NO...	El notebook SÍ...
transforma datos	carga outputs/feature_matrix_mvp.csv
entrena modelos	muestra histogramas, correlaciones, tablas
calcula métricas oficiales	permite que un revisor humano vea cell-by-cell que el resultado tiene sentido
genera CSVs	documenta narrativamente lo producido

Patrón profesional: 'notebooks as views, not controllers' (analogía MVC del desarrollo web). El controller es `src/build.py`, la vista es el notebook, el modelo son los CSVs/YAMLs.

Test crítico: si eliminás el notebook entero, el pipeline sigue funcionando (`make fase5-all` produce los outputs igual). Si eliminás `src/`, todo se rompe. Eso te dice quién es el motor y quién es el visualizador.

4. Cómo se conectan: el flujo de datos

Las seis carpetas no operan aisladas. Forman un sistema con flujos **direccionales**: los datos viajan en una sola dirección, lo cual garantiza que el sistema sea fácil de razonar. Ningún output modifica su input.

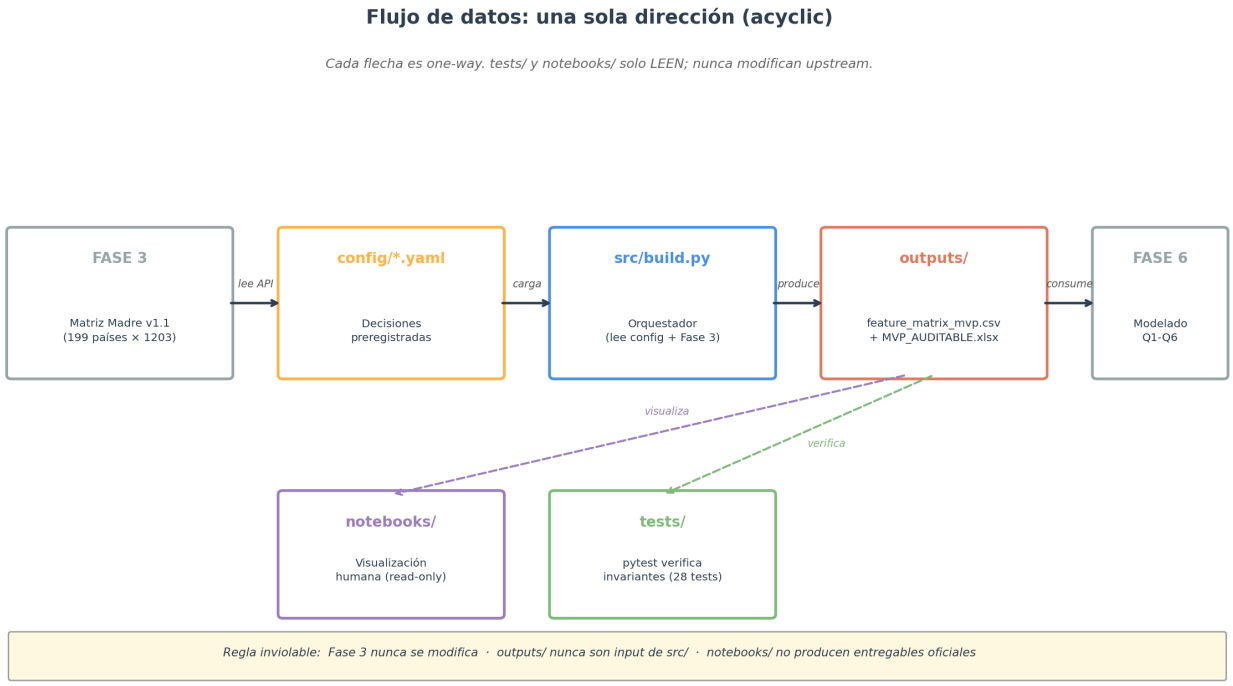

Figura 6. Flujo de datos en Fase 5. Las flechas sólidas representan el pipeline principal; las punteadas son lecturas de tests/ y notebooks/ que NO modifican upstream.

El principio: dependencias acíclicas

Esto se llama **Acyclic Dependency Principle** (R. Martin, 2017). Cada flecha es **one-way**. Si tuvieras `outputs/` modificando `src/`, o `notebooks/` modificando `config/`, tendrías un sistema imposible de razonar (un cambio podría tener efectos circulares impredecibles).

Reglas inviolables del flujo

- Fase 3 nunca se modifica desde Fase 5 (read-only via API)
- outputs/ nunca son input de src/ (eso sería *data leakage* entre runs)
- notebooks/ no producen entregables oficiales (solo visualizan outputs/)
- tests/ nunca modifican datos (solo verifican)
- config/ es la única fuente de decisiones (no hardcoded en Python)

5. El algoritmo paso a paso

Ahora que entendés cada pieza, veamos cómo funciona la ejecución completa. Cuando alguien corre `python -m src.build` (o equivalente), pasan exactamente estos 10 pasos:

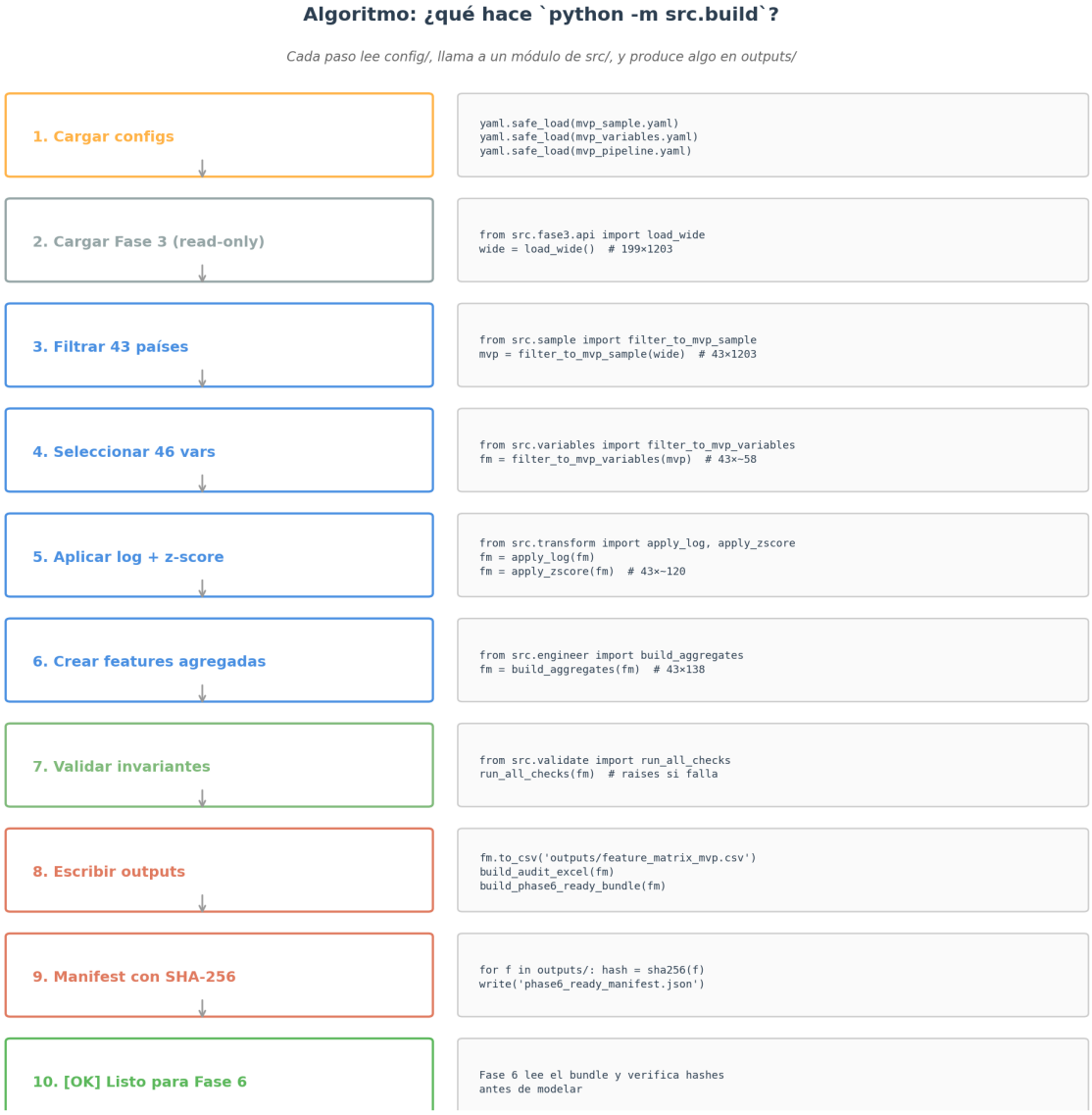

Figura 7. Algoritmo de ejecución de Fase 5. Cada paso lee un config, llama a un módulo de src/, produce algo intermedio, y lo pasa al siguiente paso.

> **Lo importante de este algoritmo:** cada paso es **determinístico** (misma entrada → misma salida) y **verificable** (existe un test que lo cubre). Si corrés `python -m src.build` dos veces, obtenés exactamente el mismo manifest hash. Eso es reproducibilidad real.

6. Checklist de buenas prácticas (resumen accionable)

Si querés aplicar estos principios en tu próximo proyecto de ciencia de datos, esta es la lista mínima:

Estructura de carpetas

- Crear `config/`, `src/`, `tests/`, `outputs/`, `notebooks/` desde el día 1
- Tener un `README.md` que explique cómo correr el proyecto
- Usar `pyproject.toml` para declarar dependencias

Configuración (config/)

- Mover decisiones a YAML/JSON (no hardcoded)
- Documentar el `razon` de cada decisión
- Versionar cambios de configuración con tags git

Código (src/)

- Una responsabilidad por archivo (Single Responsibility Principle)
- Crear un `api.py` que sea el contrato público
- Evitar mutación: funciones que reciben input y retornan output, sin side effects
- Nombres explícitos: `filter_to_mvp_sample()` > `process()`

Tests (tests/)

- Al menos un test por invariante crítico del proyecto
- Tests rápidos (< 5s total) para correr en cada commit
- Tests de inmutabilidad: verificar que NO se modifica lo que no debería
- Tests que documentan supuestos: 'no imputamos', 'mantenemos N=43', etc.

Outputs (outputs/)

- Manifest con SHA-256 de cada archivo entregado
- Backup pre-cambio antes de updates mayores
- Separar formato máquina (CSV/Parquet) y formato humano (Excel/PDF)
- Tag git inmutable cuando se cierra una versión

Notebooks (notebooks/)

- Solo lectura: notebooks NUNCA producen outputs oficiales
- Cada notebook ejecutable de cero (Restart & Run All) sin error
- Si una celda hace lógica importante, mover esa lógica a `src/`

Trazabilidad git

- Commits pequeños con mensajes descriptivos (qué + por qué)
- Tags para versiones cerradas (`fase5-v1.0`, `fase5-v2.0`)
- No commitear datos sensibles (.env, credenciales)
- `.gitignore` para outputs grandes (usar DVC si crece)

7. Referencias profesionales

Esta arquitectura no es invención propia: refleja estándares establecidos en la industria y la academia. Si querés profundizar:

Cookiecutter Data Science (Drivendata). El template estándar de la industria para proyectos de ciencia de datos. Tu Fase 5 sigue casi 1:1 esa convención.
https://drivendata.github.io/cookiecutter-data-science/

Hidden Technical Debt in Machine Learning Systems. Sculley et al., NIPS 2015. Paper seminal de Google sobre por qué la separación de concerns es crítica en ML.

Twelve-Factor App. Heroku. Principios de configuración, reproducibilidad y dependencias. Especialmente relevante el factor III (Config) y el VIII (Concurrency).
https://12factor.net/

Clean Code & Clean Architecture. Robert C. Martin. Single Responsibility Principle, Acyclic Dependency Principle. Aplicables tanto a software como a pipelines de datos.

CRISP-DM. Cross Industry Standard Process for Data Mining. El proceso estándar de minería de datos que tu proyecto sigue. Cada fase del CRISP-DM = una sub-carpeta autocontenida en tu repo.

Test pyramid (Mike Cohn). 'Succeeding with Agile', 2009. La idea de tener muchos tests unitarios rápidos en la base, pocos end-to-end en la cima.

PEP 8 + PEP 517/518. Convenciones de Python para estructura de paquetes (`pyproject.toml`, `src/__init__.py`).

DVC (Data Version Control). Para proyectos donde los outputs son demasiado grandes para git. Permite versionar datos como se versiona código.
https://dvc.org/

MLflow. Plataforma open-source para tracking de experimentos, registro de modelos, y deployment. Útil cuando el proyecto crece.
https://mlflow.org/

> **Mensaje final:**
>
> Esta arquitectura no es 'sobre-ingeniería' para un proyecto chico — es lo mínimo que un proyecto de ciencia de datos profesional necesita para sobrevivir al paso del tiempo, al cambio de personas, y a la auditoría metodológica.
>
> El mejor momento para implementar estas prácticas es **desde el día 1**. El segundo mejor momento es ahora.